# S P E C T R A V I D E O

# U S E R S  G R O U P  -- Wgtn

Newsletter Number 9 -- April 1985

---

The next meeting is on Monday ,April 22nd at 7.30 in the Staff Common Room , Wellington Clinical School , Mein Street Newtown.

The following meeting is on May 20th , same place and time.

---

## Wgtn SV Users Group - - - - -

Secretary / Newsletter Editor

Don Stanley Ph 896379 (hm) , 746906 (work)

Treasurer

Colin Chin Ph 325656 (hm)

Committee members

| | |
|---|---|
| Garry Clark | 367872 |
| Ross Wakelin | 793948 |
| Dennis Timmons | 837200 |
| Darryl Alison | 85522 (Paraparaumu) (hm) 399510 (wk) |

Specific areas of responsibility for committee members are ... Don -- secretary & newsletter ; Colin -- Treasurer & MSX developments ; Garry -- Radio Access & public domain tape software ; Ross -- National Committee ; Dennis -- library ; Darryl -- program evaluation committee chairman & Kapiti users rep

---

## New members - - -

Welcome to ... Caryl Hamer , Barry & Sonia Stubbs , John Grylls , Peter Cropp , Clive Smith , M D Bramald ,John Seelye , Glenn Baumann .

Could any members not having received a membership card yet please let me know . Also note that expiry date for subscriptions is printed on the address label of newsletter.

## Number of Members = 124

## Summary of last meeting

Lester Reilly from the CHCH Users Group spoke about the CHCH group which is a fairly new addition to user group ranks. This group is still finding its feet in some ways , and Lester indicated few members are disk users so most expertise is in cassettes.

John Matthews demonstrated the Coleco Games Adapter , there are some excellent games running under this system , but I believe the adapter is not being sold in NZ .

On the dealers side , both dealers indicated that they still had some hardware available at the February sale price. Both are hopeful that new software will arrive shortly .

Anyone having trouble with their cassette player , Garry Clark will check this out if you can come to meetings.

A short tutorial was given before the meeting proper on handling strings in Basic. This presented the major string functions and how to use them , plus a few comments about how strings use memory in the SV.

On the library side we now have access to photocopy gear on the premises , so any copyable library articles can be done on the spot at 6c per copy.

---

## Newsletter Contributions

If you are submitting programs to go in the newsletter they MUST satisfy the following conditions or they will NOT be printed.

1 ... be typed/printed on A4 paper with about 3 lines left blank at top and bottom of page and an inch at least around the sides

2 ... typed with a dark ribbon OR printed using emphasized print mode

3 ... include a description of what the program does , typed on A4 paper with example input

4 ... In the case of programs above 20 lines in length they should be sent on cassette , in this case there is no need for the printout as I can merge them into the newsletter. Enclose a stamped addressed envelope for return of cassette

5 ... line numbers should be continuous -- do a RENUM before printing the programs

6 ... PLEASE MAKE SURE THEY WORK . and would readers please note

that I am NOT responsible for programs sent by people which do not work .

7 ... In the case of screen dumps from 3d plotters please indicate the function used and try to get the print as dark as possible -- not always easy i know .

All other newsletter contributions should be sent to me at P.O Box 7057 , Wellington South. They may miss being printed for a month if sent to any other box or given to anyone else. Articles should follow 1 and 2 above.

Deadline for Newsletter articles is the 30/31st of each month. Please do not send articles direct to the printer.

---

Discounts for members .. All discounts offered by Einstein Scientic and Microstyle Computers do NOT include hardware. Discounts may be offered on such peripherals as floppy disks, cassettes , books , furniture, printers , monitors but NOT on SpectraVideo hardware. Individual dealers may not offer all the goods above at discounted rates , inquire from the shops just what discounts are available to club members. Note that to obtain a discount on any goods you must produce your current membership card.

```
*****************************************
*          E D U C A T I O N A L        *
*  S O F T W A R E   S O F T W A R E    *
*        C O M P E T I T I O N          *
*****************************************
```

Just a reminder of this competition.
The closing date has been slightly extended so that all the other computer groups have a fair chance in getting in their entries.

The new closing DATE 6th MAY 1985.
Judging is to be done by Radio Access staff who are not computer users. The instructions should be wihin the program and very user friendly! I would think that they would like to be educated, on any subject, in the nicest possible way.
Don't forget this is just about a challenge between computers with all the other brands taking part.

Address your cassettes to;
Access Radio, Computer Programs, Box 2396 , Wellington.
Dont for get to put your name and address on the cassette.

## Little things ---

The SV modem was submitted to the Post Office and rejected on several minor points. It is in the process of being upgraded to the PO requirements and CDL are hopeful of it being released within the next month. This will be a 300 Buad modem , not 1200 as has been suggested.

CDL suggest that users with software for sale now send it straight to them to go on to SpectraVideo .

If you are of a sufficiently unusual mind to attempt to use the circle command with a NEGATIVE radius you will find that the system will lock up without drawing any circle and only a CNTRL/STOP will return you to Basic.

ON ERROR GOTO <wherever> is NOT automatically reset by BASIC at the end of a program. Thus you finish a program , type in some garbage and suddenly find yourself back in the program at your error routine. Always do an ON ERROR GOTO 0 at the end of a program to ensure error trapping is reset properly.

An unseen cursor exists in the SV for graphics. Its value is always the pixel at which the next graphics operation will start. Its value can be found by peeking FE42/46 for the horizontal pixel and peeking FE44/45 for the vertical pixel. Note that the CIRCLE command leaves this graphics cursor at the CENTRE of the circle , not the last point drawn in the circle. All other commands (LINE , PSET , POINT , PRESET , PAINT ... ) leave the graphics cursor at the last point drawn.

Locations F7F6 to F7F6+25 contain the default variable types for each letter of the alphabet. The first location contains the default for variables beginning with A , the second for B etc. If no DEFINT/STR/SNG/DBL commands are given in your program then these are 8 (meaning 8 bytes for double precision) . DEFINT A - Z would see these 26 locations filled with 2 (integers need only 2 bytes) , DEFSTR would fill them with 3 (string variables need 3 bytes -- see further on) and DEFSNG would fill them with 4 .

String variables use memory as follows. 3 bytes are used for information about the string. The first byte is the string length , the remaining two give the memory location where the actual string may be found. The 3 bytes are not normally contiguous to the actual string. The statement VARPTR(a$) returns the bytes where the string length is stored , the next two bytes are the pointer to the actual string.

## GAMES SCORES

| | | |
|---|---|---|
| Spectron | 9700 | Alexander Lindsay |
| Sector Alpha | 666400 | Gavin Knight |
| Kung Fu | 38170 | Taane Clark |
| | 17690 | Garry Clark (*) |
| S C Force | 748600 | Deirdre Chin |
| Telebunny | 87810 | Colin Chin |
| | 26470 | Don Stanley (+) |
| Omac | 35360 | Richard Bremford |
| | 25700 | Don Stanley (+) |
| Ninja | 64200 | Nicola Tuinman |
| Tetra | 266600 | Nicola Tuinman |
| Turboat | 68390 | Garth Thornton (*) |
| Sasa | 118240 | John Tuinman |
| Frantic Fred | 11900 | Don Stanley (level4) |

(+) = started in normal level
(*) - started in arcade level
otherwise practice level or unknown

---

## Extra Club Meetings ....

Kapiti owners ... a club meeting will be held in Paraparaumu on APRIL 13th (Saturday) from 1.00 to 4.00 . This meeting will be held in the RAUMATI SOUTH MEMORIAL HALL , TENNIS COURT ROAD , RAUMATI SOUTH. See the Kapiti Observer & Coastlands Notice Board for more details. Several members will be on hand to demonstrate hardware ,software ,answer questions, and so on.

Hutt Valley owners ... likewise on JUNE 1st same time at a venue to be advised ... Watch the newsletter and Hutt News , as well as local computer retailers for details.

So if you live out in the suburbs and can't usually attend meetings heres your chance --- tell your friends !!

---

## Club Sponsers

Einstein Scientific -- 177 Willis Street , Wellington

MicroStyle Computers -- 198 Jackson Street , Petone

## Program Evaluation Committee

What is it ... a 3 person committee who will evaluate your program and return comments on it to you

Why .. to provide software writers with unbiased comments on programs, particularly if you may wish to sell it later.

Who .. Darryl Alison , Andrew McMillen , Jonathan Clark

Where .. send programs on cassette PLUS DOCUMENTATION to Darryl at P.O Box 189 , Paraparaumu , or phone him with details.

and .. this committee will also provide a program assistance service. Ring Darryl if you have problems and he will put you on to the best person to try and solve it. This service will include problems with BASIC or CP/M and its components (eg DBASE2 , Wordstar , MBasic , 8080 Assembler , Pascal etc ). If you want to be included as someone with particular skills who Darryl could advise members to contact please give him your name , phone number and what you can help with .

When sending programs to Darryl please include details of what the program should do , whether it is disk/cassette/either , give a name for it , and whether you intend selling it commercially later , printing it in a magazine (Bits & Bytes etc) or whatever.

---

## CP/M DISK SPIN OFF PATCH

This patch has only been tried on an SV328 Mark One with CPM2.22. It will patch the BIOS to stop the disk spin immediately , as for the patch for disk basic I had in some time ago.

Boot CP/M and carry out the following on a spare sysgened CP/M disk in drive A. You will need the programs DDT.COM (or DDT80) and SYSGEN.COM on the disk. Follow the following instructions ...

Type *DDT SYSGEN.COM* ,press ENTER , and then *G100,1A4*. At this stage you will be prompted by SYSGEN for the drive which contains the CPM system tracks. Enter drive letter (A or B) and then press enter in response to the DESTINATION ON ? , PRESS ENTER prompt from sysgen. Sysgen will run up to the breakpoint set by the G above , ie 1a4. When the DDT prompt reappears enter *S2838*. This puts you into DDT's byte changing mode , there should be a 21 on the screen now , with the cursor beside it , PRESS ENTER. There should now be an *08* with the cursor beside it , enter the number 32 and press enter. There should now be an *07* with the cursor beside it , enter 00 and press enter. Now enter a period ( . ) and press enter. Type *G* and follow the instructions from sysgen . The system will reboot as usual with sysgen , then hopefully the drives will switch off virtually immediately. If not then somet-

hing went wrong with the above procedure , which was why you did it on a spare disk. So re sysgen that disk as usual and try again.

Now for the bad news , COPY will not work with this patch. Now for the good news ... the following patch will make COPY work after the above patch has been done . (Thanks to Australian Users Group for this) . This applies to COPY.COM only , ICOPY.COM will still work properly.

You need DDT and COPY.COM on your disk. Type *DDT COPY.COM* and enter. Now type *S100* followed by enter and when the DDT prompt comes up change the next 3 bytes to C3,50,05 . Then enter a period and on the next line type *S550* and enter.CHange the next 3 bytes to 3E,07,32. The next 2 bytes depend on whether you have CPM2.20 or CPM2.22 (CPM will tell you when you boot the system). For CPM2.20 change the next 2 bytes to 70,EF. For 2.22 change them to 50,F0 .

Now change the next 6 bytes to 31,63,05,C3,03,01 and then enter a period . All the changes have now been made , so press ^C (control and C together) which will return you to CPM. At the A> prompt enter SAVE 5 COPY2.COM . Now you still have the original COPY.COM and also a file called COPY2.COM which is just COPY.COM with the above patch. Try out COPY2.COM by typing COPY2 , it should work exactly as COPY does and make a copy of your disk. If not then repeat the above procedure carefully .

---

## BUGS BUGS BUGS . . . .

Here are some more bugs from the bugs file ...

1 ... BUG --- THE DRAW COMMAND WILL NOT ALWAYS ACCEPT AN ANGLE 1 OR 3. AN ILLEGAL FUNCTION CALL IS GENERATED .

E.G , DRAW "L15A1L40A3" .... BOMBS

FIX -- TRY TO AVOID THE USE OF A1 AND A3. IF ANYONE KNOWS UNDER WHICH CIRCUMSTANCES A1 AND A3 WORK PLEASE TELL US .

---

2 ... BUG --- THE CIRCLE COMMAND DOES NOT BEHAVE AS THE MANUALS SAY WHEN JOINING THE CENTRE TO THE CIRCLE .

EG . SEE NEWSLETTER NUMBER 6 , PAGE 2

FIX --- NONE

## Public Domain Software

1 ) Do NOT send us tapes. We will supply a standard tape which will get round the problem of people sending tapes which are too short.

2 ) The cost of tapes will be $3.00 each and we will also have a charge of $2.50 to cover handling and return postage. This is charged for each separate lot of copying done , not per tape.

3 ) Any of the PD Software may also be broadcast on Radio Access in our 5 minute slot on the first Saturday of each month.

4 ) The tapes will contain combinations of cassette & disk programs , some may be set up for 80 columns. Adapt them as you wish to suit yourself.

5 ) Send orders with payment to PDS ,P.O Box 7057 , Wgtn South, Wellington

6 ) Send us your public domain software for inclusion in the software library.

---

## Serial Numbers - - - -

If you want the club to keep a list of your hardware serial numbers then fill in the form elsewhere in this newsletter.

---

## OOPSIE

The function input routine presented 2 newsletters ago needs the following notes attached .

1 ... ensure the REM!!!!!!!!!!!!!!!!!!!!!!!!!!!! line occurs before any line of code that starts at memory location &h8F21

2 ... when inputting real numbers as part of the function ensure that you use 0.2 (eg) rather than .2 if numbers are between plus and minus 1.

And another oopsie was the ROM routines list in the last newsletter. By the time I used compressed print to get Deanes output onto newsletter width and then had it reduced and then had a new lot of ink in the copier ... well if you want an A4 size copy we can photocopy it at meetings . By the way could newsletter contributors please leave a gap at least 1 inch around the page ...

## THANKS

Theres a lot of people who work behind the scenes to help get the magazine ready each month. Heres some acknowledgement for the printers at broadcasting who are so prompt with printing when they get the newsletter , to Karen who helps me label them and staple them , to Dennis who posts them out from the hospital and Garry who races around town collecting them and returning them to me. Thanks folks .

And a word of thanks to these people who let our committee turn their residences upside down at committee meetings. Thanks again.

---

## BASIC NOTES -- DEF USR

This Basic instruction is the means of getting Basic to fire up a machine code routine. The following locations are used by DEF USR.

Locations F52B to F53F are used to tell Basic where routines are. F52B/F52C contain the location at which the first (number 0) DEF USR points to. F52D/2E contain the address where the second USR call begins and so on. Note that if you peek these you need to reverse the bytes to get the correct address. All these locations are initialised with the value 09FE , which is a jump to the routine that outputs ILLEGAL FUNCTION CALL and returns you to Basic.

Location F793 contains the type of parameter being passed to the machine code routine , or back from it. A value of 2 means an integer is passed , 3 means a string , 4 means a single precision number and 8 is a double precision number.

Locations F923 -- F92A are used to contain the parameter being passed , except in the case of a string where the 3 bytes which tell the system about the string are passed. Integer values are passed in location F925/F926 , the lower order byte is in the first. These two locations also hold the string descriptor if a string is passed. Real and Double parameters start in F923 and occupy 4 and 8 bytes respectively.

To pass values back to Basic you simply fill the above locations as required.

It is possible to add new routines to Basic via the DEF USR command. They cannot (easily) be given a name , but you just use USR to call the routine. An example of a routine to add a function to convert a string to uppercase is presented in the Machine Code Corner further on in this newsletter.

## Software Competition

There are 2 separate competitions being run at present , one by Radio Access and one by us. The Radio Access one is not restricted to SV software and will be judged by Radio Access personnel. All computers which have programs broadcast on Saturday mornings are eligible for this competition. For details of prizes listen to Radio Access on Saturday mornings. This competition closes on 6th May 1985. All software submitted for this competition must be of an educational variety.

Now the competition we are running .

1 ) programs must be educational

2 ) entrys close on 20th May 1985

3 ) All SV entries received for the Radio Access competition will also be entered in this competition

4 ) First Prize

--- SECTOR ALPHA , by courtesy of MICROSTYLE COMPUTERS
--- 2 Years subscription to Bits & Bytes
--- 2 Years Users Group Subscription

Second Prize

--- $50 discount on any purchase of $100 or more from EINSTEIN SCIENTIFIC
--- 1 Year subscription to Bits & Bytes
--- 1 Years Users Group Subscription

5 ) All programs received will become part of the WGTN Users Group Public Domain Library , except that any program the judges consider good enough for marketing will be returned to the owner to market if desired.

Thats it folks , lets get those programming cells into gear , remember the software must be educational , preferably self documenting . Judges will take into account such things as ease of understanding , ease of use , and so on. Send your SV only entries to Box 7057 WGTN , but any we get before 6th May will also go into the Access competition unless requested otherwise.

---

## Machine Code Corner

This is a new section which will look at machine code on the Spectravideo. Each month a short machine code routine will be presented complete with explanation . As usual any routines which anyone would like to have included here will be gratefully accepted.

This month to start the ball rolling here is a routine which will take a string and convert any lowercase characters to uppercase. A comparison of this routine in machine code with its equivalent in Basic is included as part of the program. A complete breakdown of the program follows the code.

```
50000 DATA  f5                  :REM push a
50010 DATA  c5                  :REM push bc
50020 DATA  d5                  :REM push de
50030 DATA  e5                  :REM push hl
50040 DATA  2a,25,f9            :REM ld   hl,(f925)
50050 DATA  4e                  :REM ld   c,(hl)
50060 DATA  23                  :REM inc  hl
50070 DATA  5e                  :REM ld   e,(hl)
50080 DATA  23                  :REM inc  hl
50090 DATA  56                  :REM ld   d,(hl)
50110 DATA  06,00               :REM ld   b,0
50120 DATA  1a                  :REM ld   a,(de)
50130 DATA  fe,61               :REM cp   61
50140 DATA  38,07               :REM jr   c,07
50150 DATA  fe,7a               :REM cp   7a
50160 DATA  30,03               :REM jr   nc,03
50170 DATA  d6,20               :REM sub   20
50180 DATA  12                  :REM ld    (de),a
50190 DATA  13                  :REM inc   de
50200 DATA  04                  :REM inc   b
50210 DATA  78                  :REM ld    a,b
50220 DATA  b9                  :REM cp    c
50230 DATA  28,02               :REM jr    z,02
50240 DATA  18,ec               :REM jr    ec
50250 DATA  e1                  :REM pop   hl
50260 DATA  d1                  :REM pop   de
50270 DATA  c1                  :REM pop   bc
50280 DATA  f1                  :REM pop   a
50290 DATA  c9                  :REM ret
50300 I1=37:I2=&HD000:
      FORI =I2TO I2+I1:READA$:POKEI,VAL("&h"+A$):NEXT
50310 INPUT"Enter a string in lowercase ";NF$:A$=NF$
50320 DEFUSR=&HD000:
      PRINT"Machine Code Routine to Convert to Uppercase --
      TIMER ON NOW":T=TIME:A=USR(VARPTR(NF$)):T1=TIME:
      PRINT"MC ROUTINE DONE IN ";T1-T;" TIME UNITS"
50330 PRINT NF$
50340 PRINT:
      PRINT"Basic Code to convert to Uppercase -- TIMER ON NOW":
      T=TIME
50341 FOR I=1 TO LEN(A$):T1=ASC(MID$(A$,I,1))
50342 IF T1>=&H61 AND T1<=&H7A THEN MID$(A$,I,1)=CHR$(T1-&H20)
50343 NEXT:T1=TIME:
      PRINT"BASIC ROUTINE DONE IN ";T1-T;" TIME UNITS":PRINT A$
```

The data statements contain the HEX code for the machine code routine , the :REM statements are the actual machine code program , this does not have to be there only the HEX code is needed in

the data statements. It is often useful to include the machine code itself , especially when debugging. The HEX codes can be found in most Z80 machine code books , I find MASTERING CPM , BY ALAN R MILLER to be extremely useful in this regard.

Lines 50000-50030 take the current values of the Z80 registers and save them in an area of memory known as the stack. This is done to ensure that our BASIC program which calls the routine does not get corrupted in any way. Later these values will be restored.

Line 50040 uses one of the locations mentioned above to obtain the address of the string. This address is entered into location F925 via the USR call in line 50320. The instruction means take the contents of F925/F926 and place them into the HL register of the Z80.

Now HL contains the location where the string descriptor is found. This is the address where the 3 bytes describing the string are. The first of these 3 bytes is the string length , and the next instruction , LD C,(HL) , fills the C register of the Z80 with the string length.

Recall HL contains the address of the start of the string descriptor which is just the string length. The next 2 bytes contain the actual address where the string is found. So we need to increment HL so it now points to the next byte , save that byte somewhere and then increment HL again so it then points at the 3rd byte of the string descriptor and save that one somewhere also. The 4 lines of code from 50060 to 50090 accomplish this , the address where the string if found is now in the DE register pair. (Z80 machine code allows the DE , BC , HL registers to be treated together or as the individual registers B,C,D,E,H,L)

Now we set the B register to be zero. This register will be added to continually as we go through the string , until its value is equal to the string length. It is going to act something like Basic's FOR .. TO .. loop.

The A register is the register where arithmetic is generally done in the Z80 , and also from which various flags are set. These flags are always on or off (binary 1 or 0) . The ones used in this program are explained further on. The LD A,(DE) instruction takes the contents of the memory location which is in DE (recall it is the start of the string) and places this byte in the A register. Thus if your string was abcd , then the A register would now contain the ascii code for a .

CP is one Z80 instruction for comparing a value with another value. CP 61 compares the contents of the A register with 61. Recall these are ascii codes so only numbers are compared at this level of programming.

If the value in A is equal to the CP value (61 above) then the ZERO flag is set. When this flag is set after a CP it means that

the result of the CP was equality between the operands (61 and the contents of the A register) . If the comparison is such that the value in the A register is less than the other operand ( A register would need to contain a value with ascii code less than 61 above) then the CARRY flag is set. Now 61 is the ascii value for a lowercase A , so the CP 61 command will set the carry flag only if the value in the A register is some character which occurs in the ascii character set before lowercase A. If this condition occurs , there is no need to try to convert to upper-case , so this knowledge of the carry will allow us to bypass part of the routine.

JR C,07 causes a relative jump IF the carry flag is set. A relative is one where is made over a certain number of bytes , in the case 7 , rather than to a specific address .

Now the next CP 7A compares the contents of the A register with a lowercase Z . The routine only gets to here if the value in A was a character which could be lowercase , so here we check that it really is lowercase. If the carry flag is set after the CP 7A , then it means that the character is less (or equal to) a lowercase Z , ie it lies between lowercase A and lowercase Z .

The JR NC,03 instruction jumps 3 bytes further on IF the carry flag is NOT set . So the following 3 bytes are only seen if we have a lowercase character.

SUB 20 means subtract (in HEX) 20 from the value in A . This is done because the ascii codes for uppercase characters are 20 less than those for lowercase characters. A SUB command actually takes the value in A and changes it to the result of the SUB. Thus after this command is executed (only if A is a lowercase character remember ) then the A register contains the uppercase equivalent of the character after this instruction.

OK we have the uppercase equivalent in the A register , but we really need it in the string. The next instruction , LD (DE),A takes the value in A and loads it into the value which the DE register points to. Remember this instruction and the SUB 20 were only done for lowercase characters.

We have finished with the current character of the string now , so we increment DE to point at the next character. This is the command which the two JR (relative Jump) commands above went to. The B register is also incremented , its value is always the number of characters of the string which have so far been looked at.

Now the value in the A register is no longer needed , so now it is overwritten with the value of the B register in the LD A,B command. (LOAD A FROM B) . This is done so a comparison can be made and certain flags set.

The next command is our familiar one CP , but with a new touch. Before we compared A register with a specific value , this time

```
10  '                        "FKDEF.BAS"
20  '                 Function Key definition Program   CPM
30  '                      by Deane Whitmore
40  '
50  '-------------------------------------------------------------------------------
60  '                        Initialize Variables
70  '-------------------------------------------------------------------------------
80  IN$=CHR$(27)+"p":NO$=CHR$(27)+"q"
90  DEF FNP$(X,Y)=CHR$(27)+CHR$(89)+CHR$(Y+32)+CHR$(X+32)
100 PRINT CHR$(12)
110 PRINT FNP$(0,25);STRING$(79,"-")
120 PRINT " Function Key Definition Program       Version 2.1       Deane Whitmore
  <1985> "
130 '-------------------------------------------------------------------------------
140 '               Display Present Function Key Definitions
150 '-------------------------------------------------------------------------------
160 PRINT FNP$(3,0);
170 FK=1:SP$=" ":R=0
180 FOR F=64030! TO 64189! STEP 16
190 PRINT TAB(20);IN$;" Function Key  ";FK;SP$;NO$;"    ";
200 PRINT STRING$(20," ");PRINT FNP$(40,R);
210 FOR K=F TO F+15:PRINT CHR$(PEEK(K));:NEXT:PRINT
220 FK=FK+1:IF FK=10 THEN SP$=""
230 R=R+1
240 NEXT
250 '-------------------------------------------------------------------------------
260 '                       Select Key to Re-define
270 '-------------------------------------------------------------------------------
280 PRINT FNP$(10,11) "Select the Key to Change. (1 - 10): Hit ENTER to QUIT";
290 INPUT FK
300 IF FK=0 THEN END
310 IF FK>10 THEN PRINT FNP$(10,10);STRING$(80," ");GOTO 280
320 '-------------------------------------------------------------------------------
330 '                        Accept New Definition
340 '-------------------------------------------------------------------------------
350 PRINT:PRINT TAB(13);"Enter the Definition of your choice. (16 chars. max.)"
360 PRINT TAB(13);"Enter an ";CHR$(34);"X";CHR$(34);" for a Carriage Return."
370 PRINT:PRINT FNP$(19,16);IN$;" Function Key  ";FK;" ";NO$;"    ";
380 LINE INPUT DF$
390 IF LEN(DF$)>16 THEN 690
400 IF RIGHT$(DF$,1)="X" THEN DF$=LEFT$(DF$,LEN(DF$)-1)+CHR$(13)
410 IF RIGHT$(DF$,1)="x" THEN DF$=LEFT$(DF$,LEN(DF$)-1)+CHR$(13)
420 '-------------------------------------------------------------------------------
430 '                     Put Definiton into Memory
440 '-------------------------------------------------------------------------------
450 ON FK GOTO 460,470,480,490,500,510,520,530,540,550
460 T=64030!:GOSUB 560:GOTO 160
470 T=64046!:GOSUB 560:GOTO 160
480 T=64062!:GOSUB 560:GOTO 160
490 T=64078!:GOSUB 560:GOTO 160
500 T=64094!:GOSUB 560:GOTO 160
510 T=64110!:GOSUB 560:GOTO 160
520 T=64126!:GOSUB 560:GOTO 160
530 T=64142!:GOSUB 560:GOTO 160
540 T=64158!:GOSUB 560:GOTO 160
550 T=64174!:GOSUB 560:GOTO 160
560 FOR F=T TO T+15:POKE F,0:NEXT
570 FOR K=1 TO LEN(DF$):POKE T,ASC(MID$(DF$,K,1)):T=T+1:NEXT
580 '-------------------------------------------------------------------------------
590 '                     Clear Bottom Half of Screen
600 '-------------------------------------------------------------------------------
610 PRINT FNP$(10,10);STRING$(80," ")
620 PRINT FNP$(10,12);STRING$(80," ")
630 PRINT FNP$(10,13);STRING$(80," ")
640 PRINT FNP$(19,15);STRING$(80," ")
650 RETURN
660 '-------------------------------------------------------------------------------
670 '                           Error Message
680 '-------------------------------------------------------------------------------
```

```
690 FOR G=1 TO 15
700 PRINT FNP$(18,18);IN$;" DEFINITION TOO LONG ";NO$:FOR H=1 TO 100:NEXT
710 PRINT FNP$(18,18);" DEFINITION TOO LONG ":FOR H=1 TO 100:NEXT
720 NEXT
730 PRINT FNP$(18,18);STRING$(30," "):PRINT FNP$(19,15);STRING$(80," ")
740 GOTO 370
```

CPM

```
10  REM                        "SETUP.BAS"
20  REM                    by Deane Whitmore
30  REM
40  REM ------------------------------------------------------------
50  FOR C=64030! TO 64189!:POKE C,0:NEXT '..... Clear Function Keys.
60  REM ------------------------------------------------------------
70  REM                     Define Function Keys
80  REM ------------------------------------------------------------
90  A$="FILES"+CHR$(13) '..................... Function Key F1
100 B$="LOAD "+CHR$(34) '..................... Function Key F2
110 C$="SAVE "+CHR$(34) '..................... Function Key F3
120 D$="LIST " '............................... Function Key F4
130 E$="RUN"+CHR$(13) '....................... Function Key F5
140 F$="? CHR$(12)"+CHR$(13) '................ Function Key F6
150 G$="EDIT " '............................... Function Key F7
160 H$="OUT 52,0"+CHR$(13) '.................. Function Key F8
170 I$="LIST ."+CHR$(13) '..................... Function Key F9
180 J$="? CHR$(12);RUN"+CHR$(13) '............. Function Key F10
190 REM ------------------------------------------------------------
200 REM                     Assign Function Keys
210 REM ------------------------------------------------------------
220 T=64030!:FOR X=1 TO LEN(A$):POKE T,ASC(MID$(A$,X,1)):T=T+1:NEXT
230 T=64046!:FOR X=1 TO LEN(B$):POKE T,ASC(MID$(B$,X,1)):T=T+1:NEXT
240 T=64062!:FOR X=1 TO LEN(C$):POKE T,ASC(MID$(C$,X,1)):T=T+1:NEXT
250 T=64078!:FOR X=1 TO LEN(D$):POKE T,ASC(MID$(D$,X,1)):T=T+1:NEXT
260 T=64094!:FOR X=1 TO LEN(E$):POKE T,ASC(MID$(E$,X,1)):T=T+1:NEXT
270 T=64110!:FOR X=1 TO LEN(F$):POKE T,ASC(MID$(F$,X,1)):T=T+1:NEXT
280 T=64126!:FOR X=1 TO LEN(G$):POKE T,ASC(MID$(G$,X,1)):T=T+1:NEXT
290 T=64142!:FOR X=1 TO LEN(H$):POKE T,ASC(MID$(H$,X,1)):T=T+1:NEXT
300 T=64158!:FOR X=1 TO LEN(I$):POKE T,ASC(MID$(I$,X,1)):T=T+1:NEXT
310 T=64174!:FOR X=1 TO LEN(J$):POKE T,ASC(MID$(J$,X,1)):T=T+1:NEXT
320 REM ------------------------------------------------------------
330 REM          Display Assigned Values of Function Keys
340 REM ------------------------------------------------------------
350 PRINT CHR$(12);:PRINT TAB(12);CHR$(27)+"p";
360 PRINT " FUCTION KEY DEFINITIONS "
370 PRINT CHR$(27)+"q":PRINT
380 PRINT TAB(10) "F1 - FILES";TAB(30) "F6 - CLS"
390 PRINT TAB(10) "F2 - LOAD";TAB(30) "F7 - EDIT"
400 PRINT TAB(10) "F3 - SAVE";TAB(30) "F8 - OUT 52,0"
410 PRINT TAB(10) "F4 - LIST";TAB(30) "F9 - LIST ."
420 PRINT TAB(10) "F5 - RUN";TAB(29) "F10 - CLS:RUN"
430 END
```

```
10  REM                    "DISKCH.BAS"
20  REM          Routine to change logged Disk Drive
30  REM                    DEANE WHITMORE
40  REM
50  REM  Executing A$ as a machine language subprogram
60  REM  will do a call to CP/M's FDOS:
70  REM
80  REM  Select Disk is function 14 (OE Hex.)
90  REM  Call with function code in Reg. C and Parameter in Reg. E.
100 REM
110 REM  FDOS returns to the BASIC program after execution.
120 REM
130 REM ------------------------------------------------------------
140 PRINT CHR$(12);:LINE INPUT "Select the Drive. (A or B) ";X$
150 IF X$="A" OR X$="a" THEN X=0:GOTO 190
160 IF X$="B" OR X$="b" THEN X=1:GOTO 190
170 GOTO 140
180 REM ------------------------------------------------------------
190 A$=CHR$(0) '.....................................  NOP
200 A$=A$+CHR$(14)+CHR$(14) '........................  LD C,0E
210 A$=A$+CHR$(30)+CHR$(X)  '........................  LD E,X
220 A$=A$+CHR$(195)+CHR$(5)+CHR$(0) '................  JP 0005
230 REM ------------------------------------------------------------
240 A=VARPTR(A$) '.......................... Pointer to length of A$
250 A=PEEK(A+1)+256*PEEK(A+2) '............. Address of A$
260 CALL A '................................ Change logged Disk Drive
270 FILES '................................. Test for Drive selected
280 END
```

CPM

```
10  '---------      "VARPTR.BAS"       ----------
20  '---------   by Deane Whitmore     ----------
30  '
40  A$ = "     (23 spaces)          " '......... Assign A$ with a string.
50  ADDRESS = VARPTR(A$) '..................... Address holding string length.
60  LSB = PEEK(ADDRESS+1) '.................... Get LSB of address of string.
70  MSB = PEEK(ADDRESS+2) '.................... Get MSB of address of string.
80  RESTORE 200 '.............................. Restore DATA to put in A$.
90  GOSUB 150 '................................ Sub-routine to read DATA into A
100 PRINT A$; '................................. Print new A$.
110 RESTORE 210 '............................... Restore DATA to put in A$.
120 GOSUB 150 '................................. Sub-routine to read DATA into A
130 PRINT A$ '.................................. Print new A$.
140 GOTO 80 '................................... Do it all again.
150 FOR STRING = 0 TO PEEK(ADDRESS)-1 '......... Set up loop to print A$.
160 READB$ '.................................... Get character from DATA.
170 POKE (MSB*256+LSB+STRING),ASC(B$) '......... Put character into A$'s locatio
180 NEXT '...................................... Get next character.
190 RETURN '.................................... Return when all characters got.
200 DATA S,p,e,c,t,r,a,v,i,d,e,o," ",C,o,m,p,u,t,e,r,s," "
210 DATA a,r,e," ",v,e,r,y," ",c,l,e,v,e,r," ",t,h,i,n,g,s,.
```

we compare it with the contents of a register. CP C will set the ZERO flag IF the contents of A and the contents of C are the same. Recall C is the length of the string and A contains the number of characters in the string looked at so far (loaded ffrom the B register) . Note that we are only interested in the ZERO flag here , it is of no concern whether the value in A is less or greater than that in C.

JR Z,02 jumps over the next 2 bytes if the ZERO flag is set , in other words if A and C were the same , in other words if we have looked at the whole string.

The two bytes jumped over if the whole string is done with are a jump back to the LD A,(DE) instruction to look at the next character in the string. This jump is accomplished by the JR,EC instruction which is an unconditional jump , in other words if the instruction is met , the jump is made regardless of flags. When a JR is made with a jump of over 7D , the actual direction of the jump is backwards , and the number of bytes jumped is found by subtracting from 255.

The final 5 instructions are to restore the stack which we earlier saved , and to RETurn to Basic.

This program is relocateable which means that it can be placed anywhere in user ram. Relocatability is made possible by the JR instructions which save having to specify the exact location to jump to.

Putting the routine in memory is accomplished by the line of code which reads the data and pokes them into memory. Calling the program is done by the DEF USR command which tells BASIC where the routine is and by the A=USR(VARPTR(nf$)) statement which sets the routine going.

The parameter passed here is VARPTR(nf$) . This results in the location where the string descriptor begins being placed in F925.

The result of the routine is for NF$ to be converted to upper-case. For comparisons sake the equivalent in basic is also presented , along with timing information. Try this for strings of several sizes , as the string gets bigger , BASIC takes longer and longer , while the MC routine is almost constant in time up to a 255 byte long string.

---

## Color Chart

Garry Clarke has come up with this color chart as a programing aid when using color in programs. The program allows you to look at all combinations of colors with specified backgrounds.

```
10000 COLOR15,0,0:SCREEN1,3:
      PLAY"O5L9T250CDEFC":
      'COLOR Helper' by Garry CLARKMar85  C= Color
      B= Border & BackgroundE= Exit.  Curser or Stick moves
      sprite
10010 K=0:E=15:DRAW"S4A0":CLEAR
10020 X=5:Y=1:CO=4:GOSUB10760:GOSUB10580
10030 X=5:Y=25:CO=5:GOSUB10760:GOSUB10590
10040 X=5:Y=50:CO=7:GOSUB10760:GOSUB10610
10050 X=5:Y=74:CO=12:GOSUB10760:GOSUB10550:GOSUB10560
10060 X=5:Y=98:CO= 2:GOSUB10760:GOSUB10560
10070 X=5:Y=122:CO= 3:GOSUB10760:GOSUB10570
10080 X=5:Y=146:CO=10:GOSUB10760:GOSUB10550:GOSUB10540
10090 X=5:Y=170:CO=11:GOSUB10760:GOSUB10550:GOSUB10550
10100 X=132:Y=1:CO=13:GOSUB10550:GOSUB10570:X=X+2:GOSUB10760
10110 X=142:Y=25:CO=6:GOSUB10600:X=X+2:GOSUB10760
10120 X=142:Y=50:CO=8:GOSUB10620:X=X+2:GOSUB10760
10130 X=142:Y=74 :CO=9:GOSUB10630:X=X+2:GOSUB10760
10140 X=132:Y=98:CO=14:GOSUB10550:GOSUB10580:X=X+2:GOSUB10760
10150 X=132:Y=122:CO=15:GOSUB10550:GOSUB10590:X=X+2:GOSUB10760
10160 X=142:Y=146:CO=14:K=1:GOSUB10540:X=X+2:GOSUB10760
10170 X=142:Y=170:CO=1:K=0:GOSUB10550:X=X+2:GOSUB10760
10180 FOR T=1 TO 32
10190 READ O
10200 Q$=Q$+CHR$(O)
10210 NEXTT
10220 SPRITE$(1)=Q$
10230 X=112:Y=38:CO=3:CB=1
10240 PUTSPRITE1,(X,Y),CO,1
10250 R=STICK(0)ORSTICK(1)
10260 IFR=1THENY=Y-9
10270 IFR=2THENX=X+9:Y=Y-9
10280 IFR=3THENX=X+9
10290 IFR=4THENX=X+9:Y=Y+9
10300 IFR=5THENY=Y+9
10310 IFR=6THENX=X-9:Y=Y+9
10320 IFR=7THENX=X-9
10330 IFR=8THENX=X-9:Y=Y-9
10340 IFX>224THENX=224
10350 IFX<1  THENX=1
10360 IFY<0  THENY=-1
10370 IFY>159THENY=159
10380 O$=INKEY$
10390 IFO$="E"THEN10530
10400 IFO$="e"THEN10530
10410 IFO$="C"THENCO=CO+1
10420 IFO$="c"THENCO=CO+1
10430 IFO$="B"THENCB=CB+1
10440 IFO$="b"THENCB=CB+1
10450 IFCO=16THENCO=0
10460 IFCB=16THENCB=0
10470 IFCB>13THENE=1
10480 IFCB<2 THENE=15
10490 COLOR,,CB
10500 LINE(136,160)-(150,168),0,BF:COLOR15:LOCATE115,161:
```

```
      COLORE:PRINT"C =";CO
10510 LINE(136,184)-(150,191),0,BF:COLOR15:LOCATE115,185:
      COLORE:PRINT"B =";CB
10520 GOTO10240
10530 COLOR 15,4,5:SCREEN0,1:END
10540 DRAW"BM=X;,=Y;C=CO;BM+1;,;G1D10F1R6E1U10H1L6;BM+1;,+2;
      D8R4U8L4":X=X+10:RETURN:REM 0
10550 DRAW"BM=X;,=Y;C=CO;BM+3;,;G2D2E2D8L2D2R6U2L2U10L2":
      X=X+10:RETURN:REM 1
10560 DRAW"BM=X;,=Y;BM+1;,;C=CO;G1D2R2U1R4D4L5G1D5R8U2L6U2R5
      E1U6H1L6":X=X+10:RETURN:REM 2
10570 DRAW"BM=X;,=Y;BM+1;,;C=CO;G1D2R2U1R4D3L2D2R2D3L4U1L2D2F
      1R6E1U4H1E1U4H1L5":X=X+10:RETURN:REM 4
10580 DRAW"BM=X;,=Y;C=CO;BM+0;,+7 ;D3R5 D2R2U2R1U2L1U8L3G1D1G1D
      1G1D1G1D1;BM+2;,;R3U6G1D1G1D1G1D1":X=X+10:RETURN:REM 4
10590 DRAW"BM=X;,=Y;C=CO;D6R6D4L4U1L2D2F1R6E1U6H1L5U2R6U2L8":
      X=X+10:RETURN:REM 5
10600 DRAW"BM=X;,=Y;C=CO;BM+1;,;G1D10F1R6E1U5H1L5U3R4D1R2U2H1L
      6;BM+1;,+7;D3R4U3L4":X=X+10:RETURN:REM 6
10610 DRAW"BM=X;,=Y;C=CO;BM+8;,+2;U2L8D3R2U1R4G1D1G1D1G1D1G1D1G1
      D1R2U1E1U1E1U1E1U1E1U1E1":X=X+10:RETURN:REM 7
10620 DRAW"BM=X;,=Y;C=CO;BM+1;,;G1D4F1G1D4F1R6E1U4H1E1U4H1L6;
      BM+1;,+2;D3R4U3L4;BM+0;,+5;D3R4U3L4":X=X+10:RETURN:REM 8
10630 DRAW"BM=X;,=Y;C=CO;BM+1;,;G1D5F1R5D3L4U1L2D2F1R6E1U10H1L6;
      BM+1;,+2;D3R4U3L4":X=X+10:RETURN:REM 9
10640 DRAW"BM=X;,=Y;C=CO;BM+3;,;G3D14F3R10E3U14H3L10;BM+1;,+5;
      D10F1R6E1U10H1L6G1"
10650 IF K=1 THEN 10660 ELSE PAINT(X+5,Y+1),CO
10660 X=X+20:RETURN:REM O
10670 DRAW"BM=X;,=Y;C=CO;BM+3;,;G3D14F3R10E3U3L4D2L7H1U10E1R7
      D2R4U3H3L10"
10680 IFK=1THEN 10690 ELSE PAINT(X+5,Y+1),CO
10690 X=X+20:RETURN:REM C
10700 DRAW"BM=X;,=Y;C=CO;D20R16U5L4D1L8U16L4"
10710 IFK=1THEN10720 ELSEPAINT(X+2,Y+1),CO
10720 X=X+20:RETURN:REM L
10730 DRAW"BM=X;,=Y;C=CO;D20R4U5R5F3D2R4U4H3R1E2U9H2L14;BM+4;,
      +4;D7R7E1U5H1L7"
10740 IFK=1THEN10750 ELSE PAINT(X+5,Y+1),CO
10750 X=X+20:RETURN:REM R
10760 GOSUB10670:GOSUB10640:GOSUB10700:GOSUB10640:GOSUB10730:
      RETURN
10770 DATA31,32,47,47,44,43,43,43
10780 DATA43,43,43,44,47,47,32,31
10790 DATA248,4,244,244,52,212,244,244
10800 DATA244,244,212,52,244,244,4,248
```

---

Product review.

## TURBO PASCAL

This is a quick product review of *TURBO PASCAL* for those who were not able to see it in operation a couple of meetings ago.

*TURBO PASCAL* is a compiled language to run under CP/M2.2x. It requires that your CPU be a Z80 so we Spectravideo users are all right.

Normally, compiled languages require that you first type your program source code in using a word processer then save that to disk. Next you load your compiler program and compile the source code. If any errors are found you then load the word processer again, fix the errors and then you are back to loading the compiler etc etc etc ad infinitum or until you give up.

*TURBO PASCAL* is different in that the compiler and word processer are in the same package. You fire up *TURBO* and you can jump straight into the word processor. When you have finished typing in your program, just a quick control k q puts you at the compiler options. If the compiler finds an error (bound to happen), the compiler stops and automatically calls the word processor option and shows you on the screen where it thinks the error is (and it is usually right!). You can then fix it straight away and go back to compiling (of course from the beginning).

The second thing about compiled languages is that they usually compile down to an intermediate language, one that is not what you typed in, but is not yet down to machine code. When the language runs, a special program INTERPRETS this intermediate code as the program runs. With *TURBO*, your program is compiled right down to machine language, with the associated speed increase.

The above are the two really good things about *TURBO PASCAL* that set it apart from most CP/M languages. *TURBO* is also an enhanced Pascal, with extended file and string handling attributes. There are also added functions and procedures to directly access memory and the I/O ports, and to call CP/M BDOS function. Of course, inline machine code is available, but this is not very clear, and not explained well in the manual.

Which brings me to the manual.
If you do not know Pascal, this manual will not teach you the language. It is a Reference manual for *TURBO PASCAL* only. There are some typos, and some things are not

explained very clearly, but is WAY better than some manuals I could name. (do you hear me SV??)

If you are doing a lot of work that needs fast processing, or perhaps a lot of input or output, need a language that is almost as efficient as assembly, but is quicker to implement and change, then I would recommend that you look at *TURBO*.

It is well named.

Oh and there is a rumour that BORLAND (the people who wrote *TURBO*) are going to produce a similar thing for Basic.

Ross S Wakelin.

## Changes to the Font editor which cut down storage room and Increase speed

```
1500 IF F$="QUIT" THEN 1600
1510 PRINT:PRINT"    LOADING THE FONT..PLEASE WAIT..    ":LOCATE 10,18
1520 OPEN F$ FOR INPUT AS #1
1530 FOR U=2048 TO 4096 STEP 1
1540 O=ASC(INPUT$(1,#1)):'INPUT #1,O
1550 VPOKE U,O
1560 NEXT U
1570 CLOSE #1
1580 MOTOR OFF

1770 OPEN F$ FOR OUTPUT AS #1
1780 FOR U=2048 TO 4096 STEP 1
1790 O=VPEEK(U)
1800 PRINT #1,CHR$(O);
1810 NEXT U
1820 CLOSE #1
1830 MOTOR OFF
```

## Changes to facilitate editing of Sprites in SPRITE EDITOR

```
2800 LOCATE 0,20:PRINTSTRING$(30,32):LOCATE 0,20:INPUT"Sprite No.";IO:IO$=SPRITE$(IO):IFRES=32TH
ENFORCY=2TO17ELSE GOTO 3000
2810 OI$=BIN$(ASC(MID$(IO$,CY-1,1))):OI$=STRING$(8-LEN(OI$),"0")+OI$
2812 FOR OX=6TO13
2815 IFMID$(OI$,OX-5,1)="1"THENLOCATEOX-1,CY:BX=OX:BY=CY:GOSUB680
2820 NEXT:NEXT
2830 FORCY=2TO17
2840 OI$=BIN$(ASC(MID$(IO$,CY+16-1,1))):OI$=STRING$(8-LEN(OI$),"0")+OI$
2845 FOR OX=14TO21
2850 IFMID$(OI$,OX-13,1)="1"THENLOCATEOX-1,CY:BX=OX:BY=CY:GOSUB680
2860 NEXT:NEXT:RETURN
3000 FOR CY=2 TO 9
3010 OI$=BIN$(ASC(MID$(IO$,CY-1,1))):OI$=STRING$(8-LEN(OI$),"0")+OI$
3020 FOR OX=6TO13
3030 IFMID$(OI$,OX-5,1)="1"THENLOCATEOX-1,CY:BX=OX:BY=CY:GOSUB680
3040 NEXT:NEXT
3050 RETURN
```

JOHN MATTHEWS !!!

## GAMES SCORES

| | | |
|---|---|---|
| Spectron | 9700 | Alexander Lindsay |
| Sector Alpha | 666400 | Gavin Knight |
| Kung Fu | 38170 | Taane Clark |
| | 17690 | Garry Clark (*) |
| S C Force | 748600 | Deirdre Chin |
| Telebunny | 87810 | Colin Chin |
| | 26470 | Don Stanley (+) |
| Omac | 35360 | Richard Bremford |
| | 25700 | Don Stanley (+) |
| Ninja | 64200 | Nicola Tuinman |
| Tetra | 266600 | Nicola Tuinman |
| Turboat | 68390 | Garth Thornton (*) |
| Sasa | 118240 | John Tuinman |
| Frantic Fred | 11900 | Don Stanley (level4) |

(+) = started in normal level
(*) - started in arcade level
otherwise practice level or unknown

---

## Extra Club Meetings ....

Kapiti owners ... a club meeting will be held in Paraparaumu on APRIL 13th (Saturday) from 1.00 to 4.00 . This meeting will be held in the RAUMATI SOUTH MEMORIAL HALL , TENNIS COURT ROAD , RAUMATI SOUTH. See the Kapiti Observer & Coastlands Notice Board for more details. Several members will be on hand to demonstrate hardware ,software ,answer questions, and so on.

Hutt Valley owners ... likewise on JUNE 1st same time at a venue to be advised ... Watch the newsletter and Hutt News , as well as local computer retailers for details.

So if you live out in the suburbs and can't usually attend meetings heres your chance --- tell your friends !!

---

## Club Sponsers

Einstein Scientific -- 177 Willis Street , Wellington

MicroStyle Computers -- 198 Jackson Street , Petone

## Program Evaluation Committee

What is it ... a 3 person committee who will evaluate your program and return comments on it to you

Why .. to provide software writers with unbiased comments on programs, particularly if you may wish to sell it later.

Who .. Darryl Alison , Andrew McMillen , Jonathan Clark

Where .. send programs on cassette PLUS DOCUMENTATION to Darryl at P.O Box 189 , Paraparaumu , or phone him with details.

and .. this committee will also provide a program assistance service. Ring Darryl if you have problems and he will put you on to the best person to try and solve it. This service will include problems with BASIC or CP/M and its components (eg DBASE2 , Wordstar , MBasic , 8080 Assembler , Pascal etc ). If you want to be included as someone with particular skills who Darryl could advise members to contact please give him your name , phone number and what you can help with .

When sending programs to Darryl please include details of what the program should do , whether it is disk/cassette/either , give a name for it , and whether you intend selling it commercially later , printing it in a magazine (Bits & Bytes etc) or whatever.

---

## CP/M DISK SPIN OFF PATCH

This patch has only been tried on an SV328 Mark One with CPM2.22. It will patch the BIOS to stop the disk spin immediately , as for the patch for disk basic I had in some time ago.

Boot CP/M and carry out the following on a spare sysgened CP/M disk in drive A. You will need the programs DDT.COM (or DDT80) and SYSGEN.COM on the disk. Follow the following instructions ...

Type *DDT SYSGEN.COM* ,press ENTER , and then *G100,1A4*. At this stage you will be prompted by SYSGEN for the drive which contains the CPM system tracks. Enter drive letter (A or B) and then press enter in response to the DESTINATION ON ? , PRESS ENTER prompt from sysgen. Sysgen will run up to the breakpoint set by the G above , ie 1a4. When the DDT prompt reappears enter *S2838*. This puts you into DDT's byte changing mode , there should be a 21 on the screen now , with the cursor beside it , PRESS ENTER. There should now be an *08* with the cursor beside it , enter the number 32 and press enter. There should now be an *07* with the cursor beside it , enter 00 and press enter. Now enter a period ( . ) and press enter. Type *G* and follow the instructions from sysgen . The system will reboot as usual with sysgen , then hopefully the drives will switch off virtually immediately. If not then somet-

For the sake of consistency, it's probably a good idea to modify the ribbon cable connections to your present drive, at the drive end. After extracting the drive from its case, identify wire 17 in the cable from the controller (it probably goes to pin 14 on the drive pcb edge connector). Unsolder (or cut) it and reconnect it, extended if necessary, to pin 32 of the drive pcb edge connector. Check the cable continuity from the controller end right through to the drive connector and, if satisfactory, reinstall the drive in its case. You can now test your modified system (see later).

## SV-605 Modification

You will need the following new parts:-

1 off BC547B transistor (a BC548 or BC549 might do, but they have lower voltage ratings)

1 off 3.3 kohm 1/4 watt resistor

The procedure is as follows:-

(a) remove the outer cover of the Expander and any disk drives already installed.

(b) check that the Floppy Disk Controller chip installed on the pcb (the 40-pin device) is an MB8877A. If so, you can proceed. If not, you have one of the very early SV-605s, which have a different pcb and to which an extra 74LS74A has to be added. Unfortunately for you, the modification details for this interim SV-605 pcb have not been worked out - sorry about that.

(c) looking from the front of the SV-605, identify the positions marked on the pcb for transistor T3 and resistor R20 in the rear left corner of the main pcb. (The expansion slot pcb obstructs the under-side of the main pcb in this area, so the new components have to be soldered in place from the top of the board). Carefully solder the new parts into their correct locations as T3 and R20, ensuring that the transistor is installed the right way around.

(d) check that your work is correct (T3's collector should connect to pin 32 on the drive pcb edge connector) and then reinstall the disk drive(s). Refit the Expander's cover.

## Testing

You can now reassemble and test your modified system. Try both read and write exercises and fill up the disk, keeping track of progress by monitoring the disk directory. If all proceeds well, you're away with a disk controller that can cope with double-sided disk drives! If, however, the drive makes odd noises, you've probably left a solder-bridge somewhere in the controller. Check your work on the controller pcb carefully before you go any further.

Good luck with your double-sided future!

John Bridson

## SV-801 DISK CONTROLLER

During the investigation that led to a means of modifiying the SV-801 Disk Controller for double-sided operation (see separate item elsewhere), I discovered that my own module had been modified by SV before delivery in a way that was not consistent with good circuit design. This modification results in two TTL output chips passing far more current than they are rated for, particularly if you are already using two disk drives. It may exist in other SV-801 modules and should be removed, irrespective of whether you intend to modify the module for double-sided operation.

To check your SV-801 module, there is no alternative but to disassemble it and remove the printed circuit board. On the end of the board near the drive connectors, identify IC7 and IC9 (either 7437 or 7438 devices - mine were 7438s). Turn the board over and check whether or not there are any 150 ohm resistors soldered from pins 3, 6, 8, 11 of IC7 and IC9 to the + 5 Volt line (pin 14 of IC7 and IC9). If there are, remove them, being careful not to leave behind any solder bridges between adjacent IC pins.

After making a final check that all is in order, you can now reassemble the module and put your system back together. No special testing is required to confirm proper operation.

For the technically-minded, the 8 resistors referred to were connected to the output pins of the four line-driver gates in each chip, presumeably to act as "pull-up" resistors for these particular disk drive control lines. It just happens that these lines are already terminated by 150 ohm resistors in each disk drive anyway and the extra components are thus unnecessary, as well as increasing the current flowing in the output stage of each gate. When two drives are connected, the line current with the resistors installed is well above the maximum recommended for the particular devices - hence the need to remove the resistors.

John Bridson

SPECTRAVIDEO SOFTWARE MEMORY EXPANSION PROGRAM - Cassete

Reviewed by Garry CLARK

This has got to be the cheapest way to obtain access to just under 29k of more memory - that is if you have a SV328. What this program is really about is that it enables you to hold TWO programs in the memory at the same time. Eg. One one program in the first memory bank and another in the second bank.

There are three instructions at you disposal:

CMD SWITCH , CMD COPY , CMD SWITCH STOP

To use this machine code program, first bload "Software Memory Expansion" into your computer. You cload or load your first program in and then when loaded, type in at command level the CMD SWITCH instruction and <ENTER>. This switches you out of one bank and into the second. A quick listing check will show no program resident at the present location. Another program can be loaded here.

I prefer to designate a function Key to make switching easy. eg.
KEY3,"cmdswitch"+CHR$(13) <ENTER>
On one bank I use as above and for the other bank I use capital letters. "CMDSWITCH" All this allows you to instantly flip between programs and to know which one you are on visually.

This program could be the "peace maker" with families time sharing the computer and with different preferences in games. (Not machine code progs).
For my own use I have assembled 4 utility programs all accessed from a menu which I use for working out Sound,Color, Play and Sprite parameters. Because I don't have disc I release the disc basic memory with:- CLEAR 200,&HF500 and this gives me another 3k approx. This menued program I install in one bank while in the other I have the program I am building from scratch.
I find sprites I create are not lost switching the banks.

CMD COPY is used straight off to copy the current program into the other memory bank. It will also copy your function key descriptions as well as variables strings etc. After I have used this command the function Key label has to be put in again to avoid confusion of which bank I'm in. This command is good if you are modifing a program and need a quick check on the original.

CMD SWITCH STOP is usually used from within a program. The switch to the other bank is activated and then breaks out of program control. eg. same as STOP. The other two commands can also be used from within programs. However CMD SWITCH is the most powerful acting like the gosub routine. eg. Immediately after the switch , the first program will resume execution at the place where it left from.(provided a switch is made back from the second) Variables can be exchanged through the Video Ram with the aid of Sprites.

I recommend this program and can see it will be an asset to those of us with the cassette based system. It can be used with disc systems but for those with extra RAM packs this may be a bit of a bore. The real potential of this program will probably turn up in future programs that use more memory than 29k.